

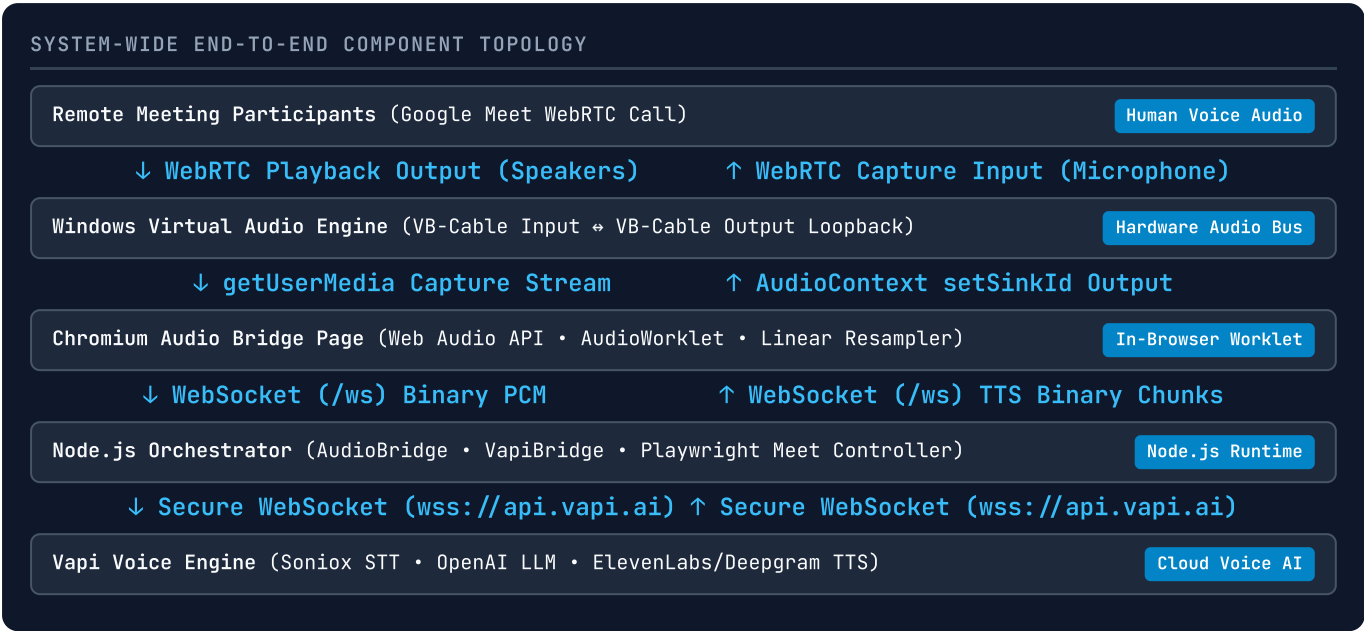
Real-Time Voice Interaction Architecture

Deep Dive into Bidirectional Audio Pipeline: Google Meet WebRTC ↔ Vapi AI Voice Agent

PROTOCOL	DUPLEX ENGINE	AUDIO WORKLET	AUTOMATION LAYER
Raw PCM WebSockets (16kHz)	Virtual Audio Cable (VB-Cable)	Linear Resampler & Hysteresis	Playwright Persistent Context

1. Executive Overview & System Architecture

Enabling a cloud-hosted conversational voice AI (Vapi) to participate in a standard Google Meet call in real time requires bridging three fundamentally isolated domains: **Google Meet's WebRTC browser sandbox**, the **Windows Host Operating System Audio Subsystem**, and the **Vapi Low-Latency WebSocket Engine**.



Pipeline A: User → Vapi (STT)

Extracts participant speech from the Meet room, resamples from Windows hardware rate (48kHz) to speech-grade PCM (16kHz 16-bit mono), performs acoustic echo suppression, and streams it to Vapi's real-time transcriber.



Pipeline B: Vapi → User (TTS)

Streams synthesized assistant PCM audio chunks from Vapi, buffers them in a low-latency AudioWorklet, resamples 16kHz → 48kHz, feeds them into VB-Cable Input, and unmutes Google Meet mic for broadcasting.

2. Pipeline A: User to Vapi (Speech Recognition Flow)

When a human participant speaks inside the Google Meet meeting, their voice traverses an 8-step ingestion and processing pipeline before reaching Vapi's Large Language Model.

1 WebRTC Audio Output in Google Meet

The remote user's voice arrives over Google's WebRTC audio peer connection. Google Meet plays the incoming audio through Chrome's default playback device. Because the agent configured the system default render device to `CABLE Input (VB-Audio)`, the user's voice outputs directly into the virtual cable.

2 Hardware Driver Loopback (VB-Cable)

The VB-Cable driver internally routes all digital audio received on `CABLE Input` directly to the `CABLE Output` capture endpoint with near-zero latency (<2ms).

3 Browser Capture via getUserMedia

The background Audio Bridge page (`http://127.0.0.1:bridgePort`) requests a microphone stream. Because Windows default capture is set to `CABLE Output`, the browser captures the human participant's audio with `echoCancellation: false` and `noiseSuppression: false` for pure studio fidelity.

4 Dynamic Linear Interpolation Resampling (48kHz → 16kHz)

Windows AudioContext runs at native hardware rate (typically 48,000 Hz or 44,100 Hz). The Audio Bridge converts 32-bit floating point audio to 16,000 Hz using linear interpolation: `ratio = inCtx.sampleRate / 16000`. This prevents audio from playing at 3× speed (chipmunk effect) in speech models.

5 PCM Encoding (16-bit Signed Little-Endian)

Normalized float samples (`-1.0` to `+1.0`) are scaled and quantized to 16-bit integers (`-32768` to `+32767`), producing raw `pcm_s16le` buffers.

6 In-Browser Echo Gating

If the assistant is currently speaking or in its trailing 500ms acoustic dissipation cooldown, the capture processor drops the chunk to eliminate self-speech loopback.

7 Local WebSocket Bridge → Node.js VapiBridge

The binary buffer is sent across `ws://127.0.0.1:bridgePort/ws` to the Node.js process. `VapiBridge.sendUserAudio()` validates the assistant state and pipes the binary frame over the persistent WebSocket to Vapi.

8 Vapi Cloud Ingestion & Speech-to-Text (STT)

Vapi's neural STT engine (Soniox/Deepgram) streams interim and final transcription tokens (`"transcript"` event). When the user pauses, Vapi's VAD (Voice Activity Detector) triggers an LLM turn.

⚠ Critical Engineering Highlight: Resampling Math

Without the custom linear interpolation resampler, 48kHz audio sent directly into a 16kHz Vapi session caused speech to be compressed by 66.7% in time, increasing pitch by 3 octaves and causing 100% recognition failure. The resampler restores natural human formant frequencies.

3. Pipeline B: Vapi to User (Assistant Speech Flow)

When the AI assistant generates a response, synthesized speech audio is streamed back into the Google Meet call with sub-second latency through the reverse pipeline.

1 Vapi Cloud TTS Audio Streaming

Vapi synthesizes conversational audio chunks (raw `pcm_s16le`, 16kHz mono) and emits a `speech-update: assistant started` message followed by binary WebSocket frames.

2 Node.js VapiBridge & Mode Switching

`VapiBridge` receives binary chunks, sets `isSpeaking = true`, and dispatches a mode change notification to the Playwright controller (`setMic(page, true)`) to unmute Google Meet's microphone.

3 AudioBridge → Browser WebSocket

Node.js forwards binary PCM chunks to the Audio Bridge browser page over the local WebSocket connection (`/ws`).

4 AudioWorklet (PcmProcessor) Queueing

The AudioWorklet thread receives the `ArrayBuffer`, converts 16-bit integers to normalized 32-bit floats, and pushes them into an internal FIFO audio buffer queue.

5 AudioWorklet Resampling (16kHz → 48kHz) & Audio Output

The worklet's `process()` callback renders 128-sample frames at 48kHz by upsampling with linear interpolation (`ratio = 16000 / 48000 = 0.333`).

6 Sink Routing to CABLE Input

The `AudioContext`'s output destination is explicitly bound to `CABLE Input (VB-Audio)` via `ctx.setSinkId(cableSink.deviceId)`.

7 Google Meet Microphone Injection

In the Google Meet page, the `MediaHook` script intercepted `getUserMedia` and forced the microphone source to `CABLE Output`. The bot's synthesized voice is ingested by Google Meet as its official microphone input.

8 WebRTC Transmission to Meeting Room

Google Meet encodes the bot's speech with the Opus codec and broadcasts it over WebRTC to all remote participants in the room.

4. Echo Prevention, Hysteresis & State Machine

Because the bot's microphone and speaker are connected through the same virtual audio driver (VB-Cable), a naive implementation creates an infinite echo loop where the bot listens to its own voice. Our multi-tiered state machine prevents this.

 **400ms Silence Hysteresis**
Streaming WebSocket audio packets arrive with 20–50ms intervals. Without hysteresis, micro-gaps between packets cause the worklet to declare `playback-stopped` prematurely, causing rapid microphone chatter. Our worklet requires **150 consecutive empty frames (400ms)** before firing a stop event.

 **Dual-Layer Echo Gate**
1. **Browser-Side:** When the worklet is playing, `isAssistantPlaying = true` drops capture buffers at the source with a 500ms trailing acoustic cooldown.
2. **Node-Side:** `VapiBridge.sendUserAudio()` drops packets if `isSpeaking = true`.

5. Protocols, Sample Rates & Latency Profile

HOP / INTERFACE	PROTOCOL / API	AUDIO FORMAT	SAMPLE RATE	TYPICAL LATENCY
Meet → VB-Cable	WebRTC Playback	PCM 32-bit Float	48,000 Hz	< 5 ms
VB-Cable Loopback	Windows Kernel Audio Driver	PCM 24/32-bit	48,000 Hz	< 2 ms
Capture & Resample	Web Audio ScriptProcessor	Linear Resampled Float → Int16	48kHz → 16kHz	~10 ms
Browser ↔ Node.js	Local WebSocket (/ws)	Binary <code>pcm_s16le</code>	16,000 Hz	< 1 ms
Node.js ↔ Vapi Cloud	WSS TLS WebSocket	Raw PCM Stream (<code>pcm_s16le</code>)	16,000 Hz	30 – 60 ms
Vapi TTS → AudioWorklet	AudioWorklet <code>PcmProcessor</code>	Upsampled Float32 Array	16kHz → 48kHz	~5 ms
Worklet → Meet Mic	Web Audio <code>setSinkId</code> → WebRTC	PCM 32-bit Float → Opus	48,000 Hz	< 5 ms

6. Summary of Key Files

- `src/audio-bridge-page.ts` : In-browser AudioWorklet, linear resampler (48k ↔ 16k), VB-Cable sink configuration, and echo gating.
- `src/vapi-bridge.ts` : Manages Vapi call session, speech state transitions, STT/TTS streaming, and padding frame filters.
- `src/audio-routing.ts` : Interacts with SoundVolumeView (`svcL.exe`) to automate Windows default playback and recording device routing.
- `src/join-meeting.ts` & `src/meet-control.ts` : Controls Playwright Chrome instance, Google Meet pre-join, in-call camera/mic state, and popup dismissals.